Hardware / Link Layer Specification

PN532 Binding of the Transport Layer Link Layer Interface (LLI)

Version 0.3 – Draft

July 24, 2026

Abstract

This document binds the abstract Link Layer Interface (LLI) defined in *smartlock_transport_layer_spec_v2.pdf* (Section 2) to a concrete controller: the NXP PN532 NFC front-end operating as an ISO/IEC 14443-4 Type A PICC in 106 kbps Passive Card Emulation Mode. It specifies the exact PN532 host commands, byte-level frame formats, and firmware-confirmed behaviors used to implement each LLI primitive. All PN532-specific claims below have been verified against NXP UM0701-02 (PN532 User Manual) and AN133910 (PN532 Application Note); none of this content should be assumed to generalize to a different controller. Should the Lock's NFC front-end ever change, only this document and its corresponding driver need to be replaced.

Contents

1 Scope & Stack Position

This document is the sole owner of PN532-specific detail in the system. It implements, and only implements, the five LLI primitives required by the transport layer: `LLI_Activate`, `LLI_ReceiveAPDU`, `LLI_SendAPDU`, `LLI_GetLinkStatus`, `LLI_Abort`. It does not define or alter APDU framing, the instruction set, protocol state machine, timing policy (T_{HS}), or status word semantics – those remain normative in the transport layer document.

The LLI sits directly above the PN532 driver and below the transport layer. It abstracts the PN532 card-emulation protocol behind a clean C API that exposes no NFC controller internals. A caller works entirely in terms of APDU byte-buffers and link status queries. The LLI imports only `pn532.h`, `pn532_cmd.h`, and `pn532_i2c.h`, never leaking PN532 types through its own public header.

1.1 Target Configuration

Parameter	Value
Role	Target (PICC), Card Emulation Mode
Modulation	Type A, 106 kbps, Passive
Protocol	ISO/IEC 14443-4 (ISO-DEP)
Host Interface	SPI / I2C / HSU (deployment-specific; not further constrained here)

2 Internal State & Initialization

The internal state is managed in a single structure (`struct lli_t`) allocated once by `lli_init` and freed by `lli_deinit`. No dynamic allocation occurs after initialization completes.

The `lli_init` sequence performs the following steps in order, cleaning up in reverse on failure:

1. **I2C Setup:** Brings up the I2C bus via `pn532_i2c_create`, adding the PN532 device and configuring IRQ/RST GPIOs.
2. **Driver Bind:** Binds the transport vtable to a new driver handle via `pn532_init` (no bus traffic).
3. **Wakeup:** Sends a 0x55 burst via `pn532_wakeup` and waits 2 ms for the oscillator to start.
4. **Firmware Validation:** Uses `pn532_get_firmware_version` to validate the IC is 0x32 and logs capabilities.
5. **SAM Config:** Configures SAM for Normal mode with IRQs enabled via `pn532_sam_configuration`.
6. **Parameters:** Enables `AUTO_ATR_RES` and `ISO14443_4_PICC` via `pn532_set_parameters`.
7. **Identity Cache:** Copies card identity fields (SENS_RES, NFCID1, SEL_RES, etc.) from the configuration struct into the handle.

3 LLI Primitive Bindings

3.1 LLI_Activate() → TgInitAsTarget (0x8C)

Before initializing as a target, LLI_Activate sends GetGeneralStatus (0x04) to check the external RF field flag. If the field byte is 0x01 (reader's field already present), the function returns LLI_ERR_INTERNAL without entering card-emulation mode.

Otherwise, it builds parameters with the mode fixed to PN532_TG_MODE_PICC_ONLY (ISO-DEP card emulation only, no DEP). The ESP32 host then issues TgInitAsTarget, and the PN532 firmware autonomously handles:

- SENS_REQ response
- SDD_REQ anti-collision cascade
- SEL_REQ completion
- RATS reception and automatic ATS transmission using the configured historical bytes

This call blocks until an NFC reader activates the PN532 or the timeout elapses. ESP_ERR_TIMEOUT maps to LLI_ERR_TIMEOUT, while any other error maps to LLI_ERR_INTERNAL.

3.2 LLI_ReceiveAPDU(timeout) → TgGetData (0x86)

Input: [0xD4, 0x86] Output: [0xD5, 0x87, Status, C-APDU (up to 262 bytes)]

This implementation uses the raw command layer (pn532_send_command + pn532_receive_response) instead of the high-level TgGetData wrapper because the LLI must access the PN532 status byte to map it to distinct error codes. The PN532 handles MI-bit chaining internally, so the LLI receives the fully reassembled payload.

3.2.1 Status Byte Mapping

PN532 Code	PN532 Meaning	LLI Result
0x00	Success (data copied)	LLI_OK
0x01	Timeout – target has not answered	LLI_ERR_TIMEOUT
0x29	Target Released	LLI_ERR_LINK_RELEASED
0x02 / 0x03	CRC / Parity error	LLI_ERR_FRAME_INTEGRITY
other	Any other non-zero status	LLI_ERR_FRAME_INTEGRITY

On any non-OK return, the output length is zeroed before the function returns. As a cryptographic safeguard, the driver MUST return an error to the transport layer *without* passing partial or corrupted data upward.

3.3 LLI_SendAPDU(bytes) → TgSetData (0x8E)

This is a thin wrapper around pn532_tg_set_data. The maximum payload is 262 bytes due to PN532 limitations. It returns LLI_ERR_SEND_FAILED on any error.

While acting as an ISO/IEC 14443-4 Type A PICC, the PN532 autonomously issues an S(WTX) request to the Initiator if the host controller does not supply the R-APDU within the negotiated Frame Waiting Time (FWT). **No additional host logic is required** to trigger WTX on this controller.

3.4 LLI_GetLinkStatus() → TgGetTargetStatus (0x8A)

The driver calls TgGetTargetStatus with a 100 ms timeout and maps its response directly to lli_link_status_t:

PN532 Status	LLI Mapping
PICC_ACTIVATED (0x81)	LLI_STATUS_ACTIVE
PICC_DESELECTED (0x82)	LLI_STATUS_ACTIVE (Deselect/reselect cycle is normal ISO14443-4 behaviour where the reader pauses the session)
PICC_RELEASED (0x80)	LLI_STATUS_RELEASED
IDLE (0x00)	LLI_STATUS_RELEASED
Communication error	LLI_STATUS_ERROR (Logged at WARN)

3.5 LLI_Abort() → InRelease / Wakeup

Abort acts as a best-effort teardown sequence, always returning LLI_OK. It guarantees the RF front-end and the cryptographic session state are torn down together:

- **ACK:** Sends pn532_send_ack to abort any in-progress PN532 command.
- **InRelease:** Issues 0x52 (Tg=0x00) via the raw command layer to release all targets.
- **Wakeup:** Sends pn532_wakeup since the chip may have entered Power-Down post-release, ensuring it is awake for the next lli_activate call.

Steps 1 and 2 are best-effort; failures are logged at DEBUG but do not cause early returns.

4 Frame Size Conformance

The PN532's Short APDU support is confirmed per NXP UM0701-02 to satisfy the bounds assumed by the transport layer without reduction:

Direction	PN532-Confirmed Limit
External PCD → PN532 (as PICC)	Up to 261 bytes
PN532 (as PICC) → PCD	Up to 258 bytes

5 Error and Status Code Reference

PN532 Command	Code	Notes
TgInitAsTarget	0x8C	Response 0x8D
TgGetData	0x86	Response 0x87
TgSetData	0x8E	Response 0x8F

PN532 Command	Code	Notes
`TgGetTargetStatus`	`0x8A`	Response `0x8B`
`InRelease`	`0x52`	Response `0x53`

6 Revision History

Version	Change	Date
0.1	Initial extraction of PN532-specific content from the transport layer draft into a standalone Hardware/Link Layer document	YYYY-MM-DD
0.2	Structural adjustments and minor technical formatting corrections	YYYY-MM-DD
0.3	Incorporated LLI C API implementation details, internal state tracking, and initialization sequences from Markdown specification	July 24, 2026